

Tecnológico de Estudios Superiores de Villa Guerrero

Ingeniería en sistemas computacionales

Unidad de Aprendizaje:

Programación Web

Nombre de la práctica:

Manual de practica

Presenta:

García Mendoza Jazmín

Profesor:

I.S.C. Reynoso Elías Juan

Semestre: Octavo

Grupo:3801

LA FINCA, VILLA GUERRERO, ESTADO DE MÉXICO.

ENERO DE 2026



Practica 1 y 2. API REST

Introducción

En esta práctica se llevó a cabo la instalación y configuración de un entorno de desarrollo para aplicaciones con **Node.js**, **Express.js** y **PostgreSQL** en un sistema Linux. El objetivo fue preparar un ambiente funcional que permitiera desarrollar aplicaciones web conectadas a una base de datos relacional. Durante el procedimiento se realizaron tareas como la creación del proyecto, instalación de dependencias, configuración de variables de entorno, creación de usuarios y bases de datos en PostgreSQL, así como la verificación del correcto funcionamiento del servicio. Estas actividades son fundamentales para establecer una comunicación segura y eficiente entre una aplicación y su sistema gestor de bases de datos.

Desarrollo de la práctica

1. Crear la carpeta del proyecto

```
mkdir learn-expressjs
```

```
cd learn-expressjs
```

```
Terminal
1. Crear carpeta del proyecto
jgarcia@tesvg:~$ mkdir learn-expressjs
jgarcia@tesvg:~$ cd learn-expressjs
```

Se creó el directorio donde se almacenarán todos los archivos de la API REST.

2. Inicializar el proyecto

```
npm init
```

```
2. Crear el paquete
jgarcia@tesvg:~/learn-expressjs$ npm init
This utility will walk you through creating a package.json file.
...
Is this OK? (yes) yes
jgarcia@tesvg:~/learn-expressjs$ ls
package.json
```

Se generó el archivo package.json, que contiene la configuración y dependencias del proyecto.

3. Instalar Express

```
npm install express
```



```
4. Instalar PostgreSQL client (pg)
jgarcia@tesvg:~/learn-expressjs$ npm install pg
added 14 packages, and audited 81 packages in 5s

26 packages are looking for funding
found 0 vulnerabilities

5. Instalar dotenv
jgarcia@tesvg:~/learn-expressjs$ npm install dotenv
added 1 package, and audited 82 packages in 4s

26 packages are looking for funding
found 0 vulnerabilities
```

Se instaló el framework Express para desarrollar el servidor web.

4. Instalar PostgreSQL Client (pg)

```
npm install pg
```

Se agregó la biblioteca que permite conectar Node.js con PostgreSQL.

```
4. Instalar PostgreSQL client (pg)
jgarcia@tesvg:~/learn-expressjs$ npm install pg
added 14 packages, and audited 81 packages in 5s

26 packages are looking for funding
found 0 vulnerabilities
```

5. Instalar dotenv

```
npm install dotenv
```

```
5. Instalar dotenv
jgarcia@tesvg:~/learn-expressjs$ npm install dotenv
added 1 package, and audited 82 packages in 4s

26 packages are looking for funding
found 0 vulnerabilities
```

Se instaló la librería que permite cargar variables de entorno desde el archivo .env.

6. Crear el archivo .env

```
DB_USER=mi_usuario
```

```
DB_HOST=localhost
```

```
DB_DATABASE=mi_base_de_datos
```

DB_PASSWORD=mi_contraseña

DB_PORT=5432

```
6. Crear archivo .env
jgarcia@tesvg:~/learn-expressjs$ nano .env
jgarcia@tesvg:~/learn-expressjs$ cat .env
DB_USER=jgarcia_db
DB_HOST=localhost
DB_DATABASE=jgarcia_db_restaurada
DB_PASSWORD=123456
DB_PORT=5432
```

Se configuraron las credenciales necesarias para conectarse a PostgreSQL.

7. Crear el usuario en PostgreSQL

```
CREATE USER mi_usuario WITH PASSWORD 'contraseña';
```

Se creó un usuario para administrar la base de datos.

```
7. Conectar a PostgreSQL
jgarcia@tesvg:~$ sudo -u postgres psql
postgres=#
8. Crear usuario
postgres=# CREATE USER jgarcia_db WITH PASSWORD '123456';
CREATE ROLE
```

8. Crear la base de datos

```
CREATE DATABASE mi_base_de_datos OWNER mi_usuario;
```

```
9. Crear base de datos
postgres=# CREATE DATABASE jgarcia_db_restaurada OWNER jgarcia_db;
CREATE DATABASE
10. Conectar a la base de datos
postgres=# \c jgarcia_db_restaurada
jgarcia_db_restaurada=#
```

Se creó la base de datos que utilizará la API.

9. Crear la tabla productos

```
CREATE TABLE productos(
    id SERIAL PRIMARY KEY,
    nombre VARCHAR(100),
```

precio NUMERIC

);

```
11. Crear tabla productos
jgarcia_db_restaurada=# CREATE TABLE productos(
    id SERIAL PRIMARY KEY,
    nombre VARCHAR(100),
    precio NUMERIC
);
CREATE TABLE
```

```
12. Salir de PostgreSQL
jgarcia_db_restaurada=# \q
postgres@tesvg:~$
```

Se creó la tabla donde se almacenarán los productos.

10. Crear el archivo db.js

Se desarrolló el archivo encargado de establecer la conexión entre la aplicación y PostgreSQL mediante la biblioteca pg.

11. Crear el archivo index.js

Se implementó el servidor Express y las rutas para las operaciones CRUD.

12. Ejecutar el servidor

node index.js

Se inició el servidor de la API REST.

13. Verificar el funcionamiento

Abrir en el navegador:

The screenshot shows a web application interface for managing products. At the top, there are three product cards: 'Laptop' with a price of 15000, 'Mouse' with a price of 120, and 'lap' with a price of 2800. Each card has an 'Eliminar' (Delete) button. Below these cards are two buttons: 'Limpiar' (Clear) and 'Obtener productos' (Get products). At the bottom, there is a form with two input fields: 'Nombre del producto' (Product name) and 'Precio' (Price), followed by a 'Guardar producto' (Save product) button.

Conclusión

La práctica permitió configurar exitosamente un entorno completo de desarrollo para aplicaciones web utilizando Node.js, Express.js y PostgreSQL. Se comprobó el funcionamiento de cada uno de los componentes mediante la instalación de paquetes, la creación de una base de datos, la definición de una tabla, la administración de usuarios y la verificación del estado del servicio de PostgreSQL. Además, el uso de variables de entorno mediante el archivo .env contribuyó a una mejor organización y seguridad de la configuración del proyecto. En conjunto, esta práctica fortaleció los conocimientos sobre la integración entre el servidor de aplicaciones y la base de datos, proporcionando una base sólida para el desarrollo de aplicaciones dinámicas y escalables.

Practica 3 y 4 Angular

Introducción

En esta práctica se llevó a cabo la creación de una aplicación utilizando el framework Angular. Para ello, se utilizó Angular CLI (Command Line Interface), la herramienta oficial que facilita la creación, configuración y administración de proyectos Angular desde la línea de comandos. Durante el desarrollo se realizaron las tareas básicas necesarias para iniciar un proyecto, ejecutar el servidor de desarrollo y generar tanto un componente como un servicio, elementos fundamentales en la arquitectura de una aplicación Angular. Esta práctica permite comprender el flujo de trabajo básico para comenzar el desarrollo de aplicaciones web modernas.

Paso 1. Instalación de Angular CLI

Se inició la práctica instalando Angular CLI de forma global mediante el comando `npm install -g @angular/cli`; una vez finalizada la instalación, se verificó que el proceso se hubiera realizado correctamente ejecutando `ng version`, lo que permitió comprobar la versión instalada de Angular CLI, Node.js y demás dependencias, asegurando que el entorno de desarrollo estuviera listo para crear aplicaciones Angular.

```
C:\Users\j7226>node -v
v12.13.0

C:\Users\j7226>npm -v
10.9.2

C:\Users\j7226>npm install -g @angular/cli

added 288 packages in 31s

70 packages are looking for funding
  run `npm fund` for details

npm notice
npm notice New major version of npm available! 10.9.2 -> 11.16.0
npm notice Changelog: https://github.com/npm/cli/releases/tag/v11.16.0
npm notice To update run: npm install -g npm@11.16.0
npm notice
```

Paso 2. Creación del proyecto Angular

A continuación, se creó un nuevo proyecto denominado **my-shop** utilizando el comando `ng new my-shop`; al ejecutarlo, Angular CLI generó automáticamente la estructura completa del proyecto, incluyendo las carpetas principales, archivos de configuración, componentes iniciales y las dependencias necesarias para comenzar el desarrollo de la aplicación.



```
Counter.tsx +  
  
//se crea la carpeta  
cd my-shop  
El sistema no puede encontrar la ruta especificada.  
  
D:\>ng new my-shop  
√ Which stylesheet system would you like to use? CSS
```

Paso 3. Acceso al proyecto y ejecución del servidor

Posteriormente, se ingresó a la carpeta del proyecto mediante el comando `cd my-shop` y se inició el servidor de desarrollo con `ng serve --host 127.0.0.1 --port 8080`; este proceso compiló la aplicación y la puso en funcionamiento en la dirección <http://127.0.0.1:8080/>, además de activar el modo *Watch*, el cual detecta automáticamente cualquier cambio realizado en el código y actualiza la aplicación sin necesidad de reiniciar el servidor.

```
// se inicia servidor  
D:\my-shop>ng generate component productos  
A merge conflicted on path "/src/app/productos/productos.spec.ts".  
D:\my-shop>ng serve --open  
| Building...
```

Paso 4. Creación del componente Productos

Después de verificar el funcionamiento del proyecto, se generó un nuevo componente llamado **productos** mediante el comando `ng generate component productos`; como resultado, Angular creó automáticamente la carpeta del componente junto con los archivos **productos.ts**, **productos.html**, **productos.css** y **productos.spec.ts**, proporcionando la estructura básica para desarrollar la interfaz correspondiente.

Paso 5. Creación del servicio API

Finalmente, se creó un servicio denominado **api** dentro de la carpeta **services** utilizando el comando `ng generate service services/api`; al ejecutarlo, Angular generó los archivos **api.ts** y **api.spec.ts**, los cuales servirán para implementar la lógica de comunicación con una API o

gestionar el acceso a los datos de la aplicación, favoreciendo una adecuada separación entre la lógica de negocio y la interfaz de usuario.



Conclusión

La práctica permitió conocer el proceso básico para iniciar el desarrollo de aplicaciones utilizando Angular. Se aprendió a utilizar Angular CLI para automatizar la creación de proyectos, componentes y servicios, reduciendo considerablemente el tiempo de configuración inicial. Asimismo, se comprendió la importancia de separar la interfaz de usuario de la lógica de negocio mediante el uso de componentes y servicios, lo que facilita el mantenimiento y escalabilidad de la aplicación. El servidor de desarrollo permitió comprobar en tiempo real los cambios realizados, agilizando el proceso de programación y prueba de la aplicación. En conjunto, esta práctica constituye una base sólida para el desarrollo de proyectos más complejos con Angular.

Practica 6 respaldo de Postgres

Paso 1. Se crea la base de datos junto con un usuario, después de crea la base de datos productos e insertamos datos en la tabla

```
3. Crear usuario
postgres=# CREATE USER jgarcia WITH PASSWORD 'MiPassword123';
4. Crear base de datos
postgres=# CREATE DATABASE jgarcia_db OWNER jgarcia;
5. Ver bases de datos
postgres=# \l
6. Conectarse a la base
postgres=# \c jgarcia_db
jgarcia_db=#
7. Crear tabla
jgarcia_db=# CREATE TABLE productos (
    id SERIAL PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    precio NUMERIC NOT NULL
);
8. Insertar dos registros
jgarcia_db=# INSERT INTO productos (nombre, precio)
VALUES
('Laptop', 15000),
('Mouse', 250);
```

Paso 2. Se consultan los datos de la tabla, se crea un respaldo de la base de datos y se verifica que se creo correctamente.

```
9. Consultar datos
jgarcia_db=# SELECT * FROM;
jgarcia_db=# \q
11. Crear respaldo
postgres@tesvg:~$ pg_dump jgarcia_db > jgarcia_db.sql
Verificar:
postgres@tesvg:~$ ls
Resultado:
jgarcia_db.sql
12. Crear respaldo comprimido
postgres@tesvg:~$ pg_dump jgarcia_db | gzip > jgarcia_db-$(date +%Y-%m-%d-T%H-%M).sql.gz
Verificar:
postgres@tesvg:~$ ls
Resultado similar:
```

Paso 3. Se restaura el respaldo

```
13. Restaurar respaldo .sql

Crear la base si no existe:

postgres@tesvg:~$ createdb jgarcia_db_restaurada

Restaurar:

postgres@tesvg:~$ psql jgarcia_db_restaurada < jgarcia_db.sql
```

Paso 4. Se realizan los siguientes pasos para asegurar que todo este correcto

```
14. Restaurar respaldo .sql.gz
postgres@tesvg:~$ createdb jgarcia_db_restaurada
postgres@tesvg:~$ gunzip -c jgarcia_db-2026-06-02-T18-30.sql.gz | psql jgarcia_db_restaurada
15. Ver tablas restauradas
postgres@tesvg:~$ psql jgarcia_db_restaurada
jgarcia_db_restaurada=# \dt
jgarcia_db_restaurada=# SELECT * FROM productos;
jgarcia_db_restaurada=# \q
16. Ver usuarios
jgarcia@tesvg:~$ sudo -u postgres psql -c "\du"
17. Ver bases de datos
jgarcia@tesvg:~$ sudo -u postgres psql -c "\l"
18. Verificar servicio PostgreSQL
jgarcia@tesvg:~$ sudo systemctl status postgresql
19. Iniciar PostgreSQL
jgarcia@tesvg:~$ sudo systemctl start postgresql
20. Reiniciar PostgreSQL
jgarcia@tesvg:~$ sudo systemctl restart postgresql
21. Habilitar PostgreSQL al arrancar
jgarcia@tesvg:~$ sudo systemctl enable postgresql
```

Practica 7 Microservicios

Paso 2. Se realizan los pasos anteriores para tener dos apis independientes una productos y otra personas.

Nombre	Fecha de modificación	Tipo	Tamaño
 personas	15/06/2026 07:03 p. m.	Carpeta de archivos	
 productos	15/06/2026 06:19 p. m.	Carpeta de archivos	

Paso 3. Se realiza el enlace con ambas apis

```
$.ajax({  
  url: 'http://localhost:4000/personas',  
  type: 'POST',  
  contentType: 'application/json',  
  data: JSON.stringify(nuevaPersona),  
  success: function(resultado) {  
    alert('Registro agregado con éxito');  
    // Limpiamos los inputs del formulario  
    $('#nombre').val('');  
    $('#apaterno').val('');  
    $('#materno').val('');  
    getPeople();  
  },  
  error: function() {  
    alert('Error al conectar con el servidor.');  }  
});
```

Paso 4. Como resultado tenemos 2 apis funcionando de manera independiente

